allows the use of move-only types as both the function object and the arguments. See the following listing.

Listing 4.7. Passing arguments to a function with `std::async`

```
#include <string>
#include <future>
struct X
{
    void foo(int,std::string const&);
    std::string bar(std::string const&);
};
X x;
auto f1=std::async(&X::foo,&x,42,"hello");
auto f2=std::async(&X::bar,x,"goodbye");
struct Y
{
    double operator()(double);
};
Y y;
auto f3=std::async(Y(),3.141);
auto f4=std::async(std::ref(y),2.718);
X baz(X&);
std::async(baz,std::ref(x));
class move_only
{
public:
    move_only();
    move_only(move_only&&)
    move_only(move_only const&) = delete;
    move_only& operator=(move_only&&);
    move_only& operator=(move_only const&) = delete;

    void operator()();
};
auto f5=std::async(move_only());
```

Calls p->foo(42,"hello") where p is &x

Calls tmpx.bar("goodbye") where tmpx is a copy of x

Calls tmpy(3.141) where tmpy is move-constructed from Y()

Calls y(2.718)

Calls baz(x)

Calls tmp() where tmp is constructed from std::move(move_only())

By default, it's up to the implementation whether `std::async` starts a new thread, or whether the task runs synchronously when the future is waited for. In most cases this is what you want, but you can specify which to use with an additional parameter to `std::async` before the function to call. This parameter is of the type `std::launch`, and can either be `std::launch::deferred` to indicate that the function call is to be deferred until either `wait()` or `get()` is called on the future, `std::launch::async` to indicate that the function must be run on its own thread, or `std::launch::deferred |std::launch::async` to indicate that the implementation may choose. This last option is the default. If the function call is deferred, it may never actually run. For example:

```
auto f6=std::async(std::launch::async,Y(),1.2);
auto f7=std::async(std::launch::deferred,baz,std::ref(x));
auto f8=std::async(
   std::launch::deferred | std::launch::async,
   baz,std::ref(x));
auto f9=std::async(baz,std::ref(x));
f7.wait();
```

Run in new thread

Run in wait() or get()

Implementation chooses

Invoke deferred function

As you'll see later in this chapter and again in chapter 8, the use of `std::async` makes it easy to divide algorithms into tasks that can be run concurrently. However, it's not the only way to associate